



Paradigme de proiectare a aplicatiilor web

Curs 2025-2026

ORM - OBJECT RELATIONAL MAPPING

Conectare la BD [2]

- ▶ **Driver baze de date (ODBC)** – interfata **low level** care permite aplicatiilor sa comunice cu o baza de date utilizand instructiuni SQL.
- ▶ **ORM (Object Relational Mapper)** – ofera o interfata orientata obiect pentru comunicarea cu baza de date.
 - ▶ Utilizeaza intern drivere de baze de date. Poate fi implementat pentru unul sau mai multe drivere de baze de date
 - ▶ Majoritatea sistemelor ORM necesita cunostinte minime de SQL.

Conectare la BD

Driver baze de date
(System.Data.SqlClient)

```
/// <returns> returneaza 'true' daca instructiunea a fost executata cu success</returns>
4 references
public static bool ExecuteNonQuery(string sql, CommandType cmdType, params SqlParameter[] parameters)
{
    int result = EROARE_LA_EXECUTIE;
    using (SqlConnection conn = new SqlConnection(ConnectionString))
    {
        using (SqlCommand cmd = new SqlCommand(sql, conn))
        {
            cmd.CommandType = cmdType;
            foreach (var item in parameters)
            {
                cmd.Parameters.Add(item);
            }
        }
    }

    try
    {
        cmd.Connection.Open();
        result = cmd.ExecuteNonQuery();
    }
    catch (SqlException ex)
    {
        //salveaza exceptii in fisiere log
    }
}

return Convert.ToBoolean(result);
}
```

ORM – EntityFramework

```
0 references
public void AddComanie(Comanie c)
{
    using (var context = new DemoWebEntities())
    {
        context.Comanii.Add(c);
        context.SaveChanges();
    }
}
```

Este util ORM?

De ce?

Care erau procedurile
echivalente
in sistemele clasice?

Putem scrie o functie
unica de adaugare
pentru orice entitate?

```
1 references
public bool AddCompanie(Companie comp)
{
    return SqlDBHelper.ExecuteNonQuery(
        "INSERT INTO dbo.Companii VALUES (@Nume, @Email, @Telefon, @Adresa)", CommandType.Text,
        new SqlParameter("@Nume", comp.Nume),
        new SqlParameter("@Email", comp.Email),
        new SqlParameter("@Telefon", comp.Telefon),
        new SqlParameter("@Adresa", comp.Adresa));
}

2 references
public bool AddMasina(Masina m)
{
    return SqlDBHelper.ExecuteNonQuery(
        "insert into dbo.Masini VALUES (@DataFabricatie, @IdCompanie, @Model, @Pret)", CommandType.Text,
        new SqlParameter("@DataFabricatie", m.DataFabricatie),
        new SqlParameter("@IdCompanie", m.Companie.Id),
        new SqlParameter("@Model", m.Model),
        new SqlParameter("@Pret", m.Pret)
    );
}
```

Fig. 1. Exemple metode de adaugare
in varianta cu driver

Avantaje ORM

- ▶ Se ocupă de logica necesară pentru a interacționa cu bazele de date la nivel de obiect.
- ▶ Îmbunătățește securitatea. Instrumentele ORM sunt construite pentru a elimina posibilitatea atacurilor prin injectare SQL.
- ▶ Productivitate. Se scrie mai puțin cod când se utilizează instrumente ORM.
- ▶ **De reținut**
 - ▶ pentru aplicațiile care au nevoie de interogări cu adevărat performante, trebuie analizat ce SQL produce ORM-ul și, eventual, trebuie utilizat SQL nativ pentru a executa versiunea de SQL mai performantă.

ORM - abordari

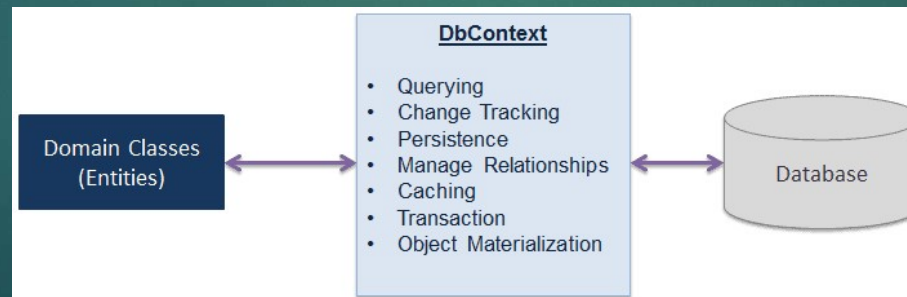
- ▶ **Schema first** – se definește prima data baza de date, cu tabele și cu relațiile dintre ele. Apoi se generează codul (modelele și contextul) pe baza schemei.
- ▶ **Code first** – în această abordare, prima dată se definește codul. Fiecare tabel corespunde unei clase și aceasta conține și informațiile despre relațiile dintre tabele. ORM-ul va genera structura SQL pe baza codului.

Concepte EF – DbContext [3]

Correspunde bazei de date sau a unui colectii de tabele din baza de date.

Responsabilitati:

- ▶ **Querying:** Converteste interogările LINQ-to-Entities la interogari SQL si le trimite bazei de date.
- ▶ **Change Tracking:** Pastreaza modificarile asupra entitatilor dupa ce au fost extrase din baza de date.
- ▶ **Persisting Data:** Efectueaza operatii de Insert, Update, Delete in/din baza de date in functie de starea entitatilor.
- ▶ **Caching:** Furnizeaza, implicit, un prim nivel de cache. Stocheaza entitatile ce au fost extrase din baza de date pe durata de viata a obiectului DbContext.
- ▶ **Manage Relationship:** administreaza relatiile intre tabele.
- ▶ **Object Materialization:** Converteste datele provenite din baza de date in obiecte entitati.



Concepte EF – DBSet [3]

- ▶ Corespunde unui tabel sau unui view
- ▶ DBSet – setul (sau colectia) de entitati care poate fi utilizat pentru operatii de creare, actualizare, citire si stergere (CRUD - create, read, update, delete).
- ▶ Clasa context (derivata din DbContext) trebuie sa includa proprietati de tip DbSet pentru entitatile corespunzatoare tabelelor si vederilor din baza de date.

```
4 references
public partial class DemoWeb_DBFirstEntities : DbContext
{
    3 references
    public DemoWeb_DBFirstEntities()
        : base("name=DemoWeb_DBFirstEntities")
    {
    }

    0 references
    protected override void OnModelCreating(DbModelBuilder modelBuilder)
    {
        throw new UnintentionalCodeFirstException();
    }

    3 references
    public virtual DbSet<Companii> Companii { get; set; }
    0 references
    public virtual DbSet<Masini> Masini { get; set; }
}
```

Metode (cateva exemple EF) [3]

Nume metoda	Descriere	Exemplu
Add	Adauga entitatea in context cu starea Added	<code>dbcontext.Companii.Add(companieEntity)</code>
AsNoTracking<Entity>	Returneaza un query in care entitatile nu sunt salvate in cache-ul implicit	<code>var listaCompanii = dbcontext .Companii.AsNoTracking<Student>() .ToList<Student>();</code>
Attach(Entity)	Ataseaza entitatea respectiva contextului cu starea Unchanged	<code>dbcontext.Companii.Attach(companieEntity);</code>
Create	Creaza o noua instanta a entitatii dar nu este adaugata sau atasata contextului	<code>var newCompanyEntity = dbcontext.Companii.Create();</code>

Scenarii de persistenta a datelor

- ▶ Persistenta unei entitati poate fi realizata in 2 moduri [3]:
 - ▶ Connected – aceeași instanță de DbContext este utilizată atât pentru extragerea cât și pentru persistența datelor (adekvată pentru aplicații standalone)
 - ▶ Disconnected – utilizează 2 instanțe diferite pentru extragere și persistența date (cazul aplicațiilor web)
- ▶ Ghid de stabilire a duratei de viață (lifetime) a contextului
 - ▶ WebApp – o instanță a contextului pentru fiecare request
 - ▶ WPF, WindowsForms – o instanță per formă
 - ▶ Contextele cu durată mare de viață (long-running)
 - ▶ -> consum de memorie
 - ▶ -> o excepție poate determina pierderea contextului
 - ▶ -> nu este thread-safe -> ar fi indicat să nu fie partajat între mai multe thread-uri care fac operații similare în mod concurrent

Modelare relatii intre tabele

► 1-To-Many – exemplu :

- `public virtual ICollection<Masina> Masini { get; set; };`
 - O lista de masini este asociata unei companii

► Foreign key – exemplu:

- `public virtual Companie Companie { get; set; }`
 - In entitatea Masina avem o referinta catre entitatea Companie

```
public partial class Companie
{
    0 references
    public Companie()
    {
        this.Masini = new HashSet<Masina>();
    }

    1 reference
    public int Id { get; set; }
    0 references
    public string Nume { get; set; }
    0 references
    public string Email { get; set; }
    0 references
    public Nullable<long> Telefon { get; set; }
    0 references
    public string Adresa { get; set; }

    1 reference
    public virtual ICollection<Masina> Masini { get; set; }
}
```

```
public partial class Masina
{
    0 references
    public int ID { get; set; }
    0 references
    public System.DateTime DataFabricatie { get; set; }
    0 references
    public int CompanieID { get; set; }
    0 references
    public string Model { get; set; }
    0 references
    public decimal Pret { get; set; }

    0 references
    public virtual Companie Companie { get; set; }
}
```

Conventii [4]

- ▶ Numele tabelelor corespund pluralului numelor entitatilor.
- ▶ Numele proprietatilor entitatilor corespund coloanelor din tabele.
- ▶ Proprietatile entitatilor numite ID sau NumeClasaID sunt recunoscute ca fiind chei primare.
- ▶ O proprietate este interpretata ca o cheie straina daca este numita:
 - ▶ <nume tabel><nume proprietate cheie primara> (exemplu: pentru tabel Companie si cheie primara ID -> cheie straina CompanieID)
 - ▶ <numele proprietatii cheii primare daca aceasta include si numele entitatii (exemplu: nume cheie primara CompanieID, cheie secundara CompanieID).

Schema First – Etape EF

- ▶ Creare bază de date si tabele - scripturi SQL sau un instrument de management al bazelor de date
- ▶ generarea modelelor pe baza structurii bazei de date
 - ▶ utilizând comenzi specifice – exemplu **Scaffold-DbContext** – disponibilă în Entity Framework Core
 - ▶ în mod vizual – utilizând componenta ADO.NET Entity Data Model disponibilă în mediul Visual Studio.
 - ▶ Wizard-ul corespunzator componentei permite
 - ▶ specificarea string-ului de conectare
 - ▶ selectarea entitatilor (tabele, view-uri etc.) pentru care va fi generat cod



Code First - Etape EF [3]

- ▶ Crearea modelelor si a clasei derivate din DbContext
- ▶ Activarea migrarilor – se genereaza clasa Configuration care contine metoda **Seed**. Aceasta metoda va fi completata cu secvente de cod pentru inserarea datelor de test in baza de date
- ▶ Generarea migrarilor – se genereaza scripturi doar pentru modificarile facute
- ▶ Aplicarea migrarilor pe baza de date - > rezulta crearea bazei de date, modificarea/crearea tabelelor si modificarea/adaugarea de noi date
- ▶ Efectuarea modificarilor
 - ▶ Se adauga noi modele sau noi proprietati modelelor existente
 - ▶ Se genereaza migrarea pentru modificarile respective
 - ▶ Se actualizeaza baza de date (se ruleaza scripturile corespunzatoare migrarilor)

```
public partial class InitialCreate : DbMigration
{
    0 references
    public override void Up()
    {
        CreateTable(
            "dbo.Companies",
            c => new
            {
                Id = c.Int(nullable: false, identity: true),
                Nume = c.String(),
                Email = c.String(),
                Telefon = c.Long(nullable: false),
                Adresa = c.String(),
            })
            .PrimaryKey(t => t.Id);

        CreateTable(
            "dbo.Masinas",
            c => new
            {
                Id = c.Int(nullable: false, identity: true),
                DataFabricatie = c.DateTime(nullable: false),
                Model = c.String(),
                Pret = c.Decimal(nullable: false, precision: 18, scale: 2),
                Companie_Id = c.Int(),
            })
            .PrimaryKey(t => t.Id)
            .ForeignKey("dbo.Companies", t => t.Companie_Id)
            .Index(t => t.Companie_Id);
    }

    0 references
    public override void Down()
    {
        DropForeignKey("dbo.Masinas", "Companie_Id", "dbo.Companies");
        DropIndex("dbo.Masinas", new[] { "Companie_Id" });
        DropTable("dbo.Masinas");
        DropTable("dbo.Companies");
    }
}
```

Migrari [3]

- ▶ O migrare este un script care fie altereaza structura bazei de date fie executa o secventa SQL care afecteaza datele (exemplu adaugarea unui set de date initiale).
- ▶ De fiecare data cand se modifica structura bazei de date, aspectele modificate trebuie sa se regaseasca intr-o secventa tranzactionala (cu actiuni de tip Up si Down) numita migrare. Migrarea este aplicata bazei de date si astfel baza de date se modifica corespunzator.
- ▶ O migrare poate fi exprimata fie utilizand cod SQL fie cod C#.

ORM – interogari

- ▶ Entity framework suporta trei tipuri de interogari [3]:
 - ▶ LINQ-to-Entities (sintaxele LINQ method si LINQ Query)
 - ▶ Entity SQL
 - ▶ Native SQL
- ▶ Incarcarea obiectelor aditionale (in cazul relatiilor) [5] se poate face prin:
 - ▶ Eager Loading
 - ▶ Lazy Loading
 - ▶ Explicit Loading
- ▶ Atentie la interogarile complexe: ar fi bine de verificat SQL-ul generat
- ▶ Alte framework-uri ORM:
 - ▶ .NET: Entity Framework, Linq2db , Dapper, Nhibernate
 - ▶ Java: Hibernate, ActiveJDBC
 - ▶ PHP: RedBeanPHP, Doctrine ORM
 - ▶ Node.js: Sequelize
 - ▶ Python: Django integrated ORM, SQL Alchemy

```
2 references
public Companie GetCompanie(int id)
{
    using (var context = new DemoWeb_DBFirstEntities())
    {
        return context.Companii
            .Where(c => c.Id == id)
            .FirstOrDefault();
    }
}

2 references
public List<Companie> GetCompanii()
{
    using (var context = new DemoWeb_DBFirstEntities())
    {
        var query = from st in context.Companii
                     select st;
        var result = query.ToList();
        return result;
    }
}
```

LINQ Method

LINQ Query

Bibliografie

1. <https://softchris.github.io/pages/dotnet-orm.html>
2. <https://www.quora.com/What-is-the-difference-between-an-Object-Relational-Mapping-ORM-and-a-Driver-for-Databases>
3. <https://www.entityframeworktutorial.net/EntityFramework6/introduction.aspx>
4. <https://www.entityframeworktutorial.net/efcore/create-model-for-existing-database-in-ef-core.aspx>
5. <https://docs.microsoft.com/en-us/aspnet/mvc/overview/getting-started/getting-started-with-ef-using-mvc/creating-an-entity-framework-data-model-for-an-asp-net-mvc-application>
6. <https://docs.microsoft.com/en-us/ef/ef6/querying/related-data>
7. <https://learn.microsoft.com/en-us/ef/ef6/modeling/code-first/migrations/>